

MemoLang

Ứng dụng cơ chế giải phóng dữ liệu đệm trong phát triển AI

LĨNH VỰC

Phần mềm hệ thống

Q1 Câu hỏi nghiên cứu & mục tiêu

- Mỗi phép nhân ma trận (GEMM) tạo ra một ma trận trung gian; trong mô hình AI, bộ đệm dồn lại rất nhanh.
- Câu hỏi:** đọc phá hủy ở cấp ngôn ngữ có giảm bộ nhớ đỉnh mà vẫn giữ tốc độ kernel không?

Mục tiêu: xây dựng MemoLang, biên dịch qua LLVM, tự giải phóng ma trận đã tiêu thụ và dùng cuBLAS cho phép nhân.

Giải thuyết

Giải phóng bộ đệm ngay khi tiêu thụ sẽ giảm bộ nhớ đỉnh và số lần cấp phát, rõ nhất với chuỗi phép nhân dài.

Q2 Phương pháp & thiết kế



■ Đã hoàn thành (C++, LLVM) □ Đã thiết kế ▨ Dùng đọc phá hủy

Đối chứng Cùng ngôn ngữ, ngữ nghĩa thông thường
Biến Kích thước ma trận, độ dài chuỗi phép nhân
Đo Bộ nhớ đỉnh, số lần cấp phát, thời gian, công suất GPU

Q3 Kết quả hiện tại & phân tích

Thông thường



7

Đọc phá hủy



3

Số bộ đệm cùng tồn tại với chuỗi $T_1 = AB \rightarrow T_2 = T_1C \rightarrow T_3 = T_2D$ (minh họa, chưa phải số đo).

Kiểm thử

Kết quả

Nhân N×N so với tham chiếu

Chưa có

Lỗi dồn lại sau tiêu thụ

Chưa có

$$T_{\text{total}} = T_{\text{lang}} + T_{\text{memory}} + T_{\text{launch}} + T_{\text{kernel}}$$

Đã chạy: biên dịch front-end cho hàm cộng cơ bản. Chưa có số đo GEMM.

Q4 Diễn giải & kết luận

- Phép nhân giao cho cuBLAS nên T_{kernel} được kỳ vọng **không đổi**; lợi ích nằm ở T_{memory} .
- Điểm mới:** khác Rust (kiểu affine) và Futhark (tự sinh kernel), MemoLang phân tích tiêu thụ lúc biên dịch và dùng thư viện GPU có sẵn.
- Giảm bộ nhớ đỉnh dù không nhanh hơn vẫn là kết quả có ý nghĩa, khi giá DRAM tăng do nhu cầu AI.
- Tiếp theo:** phân tích tiêu thụ tĩnh, kiểu ma trận, liên kết cuBLAS, rồi đo hiệu năng.